

基于 Gradio 的垃圾分类识别系统

实验文档（三）模型管理功能实现

实现步骤总览

步骤	内容	验证方式
0.4	环境自检	数据库可连、data/ 存在
1.1	import 与训练相关全局变量	py_compile 通过
1.2	路径与表格工具函数	—
1.3	数据库查询与模型登记	—
1.4	列表接口与面板状态整理	python -c 打印模型数量
2.1	上传 .pth	页面或函数测试
2.2	设置当前模型	查询 settings 表
3.1	确认训练脚本输出格式	命令行跑 1 epoch
3.2	日志解析与清洗函数	—
3.3	_train_model 子进程调度	—
4.1	扩展 build_main_panel	界面可见模型管理页签
4.2	更新 build_app 回调与登录后加载	登录后列表有数据
5.2	完整功能测试 MG1-MG8	测试表

学习路径

章节	目标
第 0 章	理解「库表 + 磁盘文件 + 当前模型配置」三者关系
第 1 章	后端：读列表、登记 checkpoint、供 UI 调用的封装
第 2 章	后端：上传与切换当前模型
第 3 章	后端：子进程训练 + 流式日志
第 4 章	前端：页签组件与 .click().then() 刷新链
第 5 章	联调测试与排错

第 0 章 功能架构

0.1 业务功能一览

说明：模型管理不是孤立模块，它与「图像识别」共用同一套权重数据源。数据库记「有哪些模型」，`settings` 记「默认用哪一个」，磁盘存真实 `.pth` 文件。

功能	界面位置	后端函数
刷新模型列表	模型管理 → 「刷新模型列表」	<code>_models_list</code> → <code>_list_models_from_db</code>
上传权重	模型管理 → 选择 <code>.pth</code> → 「上传」	<code>_models_upload</code> → <code>_register_model_record</code>
设置当前模型	模型管理 → 下拉框 → 「设置」	<code>_models_set_current</code> → <code>settings.current_model</code>
启动训练	模型管理 → 训练面板 → 「开始训练」	<code>_train_model</code> （生成器 <code>yield</code> ）
识别页选权重	图像识别 → 「权重文件」下拉	<code>_refresh_weights</code> / <code>_get_current_weight</code>

0.2 数据库与文件

`models` 表（见 `garbage.sql`）：保存权重的元信息（文件名、相对路径、架构、大小、准确率等），不保存二进制权重本身。

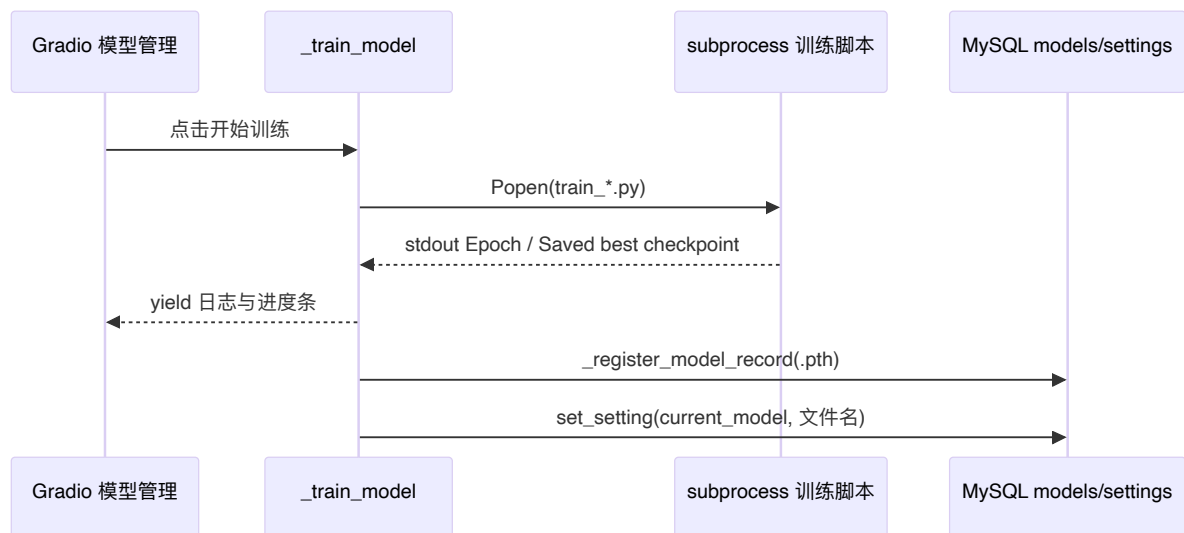
`settings` 表：键 `current_model` 的值为当前默认权重的文件名（如 `resnet50_20260422_161046.pth`）。识别页下拉框默认选中它。

磁盘目录：

```
项目根目录/
├── models/
│   ├── resnet50/           ← 训练默认输出目录
│   ├── alexnet/
│   ├── cnn/
│   └── uploaded/          ← 页面上传 .pth 的保存位置
└── train/
    ├── train_resnet50.py
    ├── train_alexnet.py
    └── train_cnn.py
```

设计要点：路径入库时用相对项目根目录的路径（`_to_relative_project_path`），换机器部署时只要整个项目文件夹一起拷贝即可。

0.3 数据流（训练完成）



0.4 环境自检（开始编码前必做）

说明：模型管理依赖登录章已写好的 `_get_db()`。若登录章未完成，请先做完登录实验文档第 0.5、第 1 章。

操作：在项目根目录执行：

```

pip install -r requirements.txt
python -c "from gradio_app import _get_db; _get_db(); print('MySQL 连接成功')"

```

确认数据集（训练时需要）：

```

# Windows
dir data\train_list.txt
# Linux / macOS
ls data/train_list.txt

```

确认 `models` 表有初始数据（可选）：

```

USE garbage;
SELECT name, arch FROM models LIMIT 3;
SELECT k, v FROM settings WHERE k = 'current_model';

```

可选：命令行试跑 1 轮训练（验证 GPU/CPU 与数据路径，约数分钟）：

```

python train/train_resnet50.py --epochs 1 --batch-size 8 --out-dir
models/resnet50 --run-name doc_test --cpu

```

检查项	通过标准
数据库	终端输出 <code>MySQL 连接成功</code>
数据集	<code>train_list.txt</code> 存在
训练脚本	最后一行含 <code>Saved best checkpoint to:</code>

第 1 章 模型元数据与列表

本章在后端建立「读库 → 拼表格 → 给 Gradio 用」的能力，暂不改界面。

步骤 1.1 补充 import 与全局变量

本节目标

引入训练调度所需的系统库与 PyTorch，并定义训练进度解析、当前模型配置键等常量。

实现说明

- `subprocess`：在独立进程中运行 `train/*.py`，避免训练阻塞 Gradio 主线程。
- `torch`：读取 `.pth` checkpoint 字典，提取 `arch`、`val_acc` 等写入数据库。
- `_EPOCH_PROGRESS_RE`：用正则从训练日志行中解析 `Epoch 3/10`，驱动进度条。
- `_CURRENT_MODEL_KEY`：与 `settings` 表中 `k='current_model'` 对应，全项目统一用该常量，避免硬编码字符串。

操作

在 `gradio_app.py` 头部合并以下 import（保留登录章已有的 `werkzeug`、`Database`、`AppConfig` 等，不要删除）：

```
import locale
import re
import shutil
import subprocess
import sys
from datetime import datetime

import torch
```

在 `_CFG = AppConfig.load()` 之后（若已有 `_DB`，不要重复定义）追加：

```
_EPOCH_PROGRESS_RE = re.compile(r"Epoch\s+(\d+)\s*/\s*(\d+)\s|"Epoch\s+(\d+)\s"/(\d+)\s")
_TRAIN_NUM_WORKERS = 2
_TRAIN_FORCE_CPU = False
_TRAIN_RESNET50_PRETRAINED = True
_CURRENT_MODEL_KEY = "current_model"
```

代码解读

变量	含义
<code>_TRAIN_NUM_WORKERS</code>	传给训练脚本的 DataLoader 线程数
<code>_TRAIN_FORCE_CPU</code>	为 <code>True</code> 时命令行追加 <code>--cpu</code> ，无显卡机房可改
<code>_TRAIN_RESNET50_PRETRAINED</code>	ResNet50 是否使用 ImageNet 预训练

验证

```
python -m py_compile gradio_app.py
```

步骤 1.2 路径与表格工具函数

本节目标

统一「项目根路径」「训练输出目录」「相对路径入库」以及把字典列表转成 Gradio `Dataframe` 需要的二维数组。

实现说明

Gradio 的 `Dataframe` 组件接收 `list[list]`，而数据库查询结果是 `dict` 或 `Row`，需要 `_items_to_rows` 做字段投影。
`_model_output_dir` 约定每种架构权重落在 `models/<架构名>/` 下，与训练脚本 `--out-dir` 一致。

操作

在全局变量下方继续追加：

```
def _project_root() -> str:
    return os.path.dirname(os.path.abspath(__file__))

def _model_output_dir(model_name: str) -> str:
    return os.path.join(_project_root(), "models", model_name)
```

```
def _to_relative_project_path(path: str) -> str:
    abs_path = os.path.abspath(path)
    root = _project_root()
    try:
        rel_path = os.path.relpath(abs_path, root)
    except ValueError:
        return abs_path
    return rel_path.replace("/", os.sep)

def _items_to_rows(items: list[dict[str, Any]], fields: list[str]) -> list[list[Any]]:
    return [[item.get(field) for field in fields] for item in (items or [])]

def _models_to_rows(items: list[dict[str, Any]]) -> list[list[Any]]:
    return _items_to_rows(
        items, ["name", "size_mb", "num_classes", "epoch", "val_acc",
                "updated_at", "is_current"]
    )
```

代码解读

- `_to_relative_project_path`: 把 `D:\proj\models\a.pth` 转为 `models\a.pth` 再写入 `models.path` 字段。
- `is_current` 不在数据库表中, 在 `_list_models_from_db` 里根据 `settings` 动态计算 (见下一步)。

验证

```
python -c "from gradio_app import _model_output_dir;
print(_model_output_dir('resnet50'))"
```

预期输出类似: `...\garbage-classifier-flask\models\resnet50`。

步骤 1.3 数据库查询与模型登记

本节目标

实现: ① 查询所有模型并标记当前项; ② 解析 `.pth` 并插入/更新 `models` 表 (训练结束与上传共用)。

实现说明

`_list_models_from_db`: 从 `db.list_models()` 取行, 与 `settings.current_model` 比对, 给每条记录加 `is_current` 布尔字段, 供表格最后一列显示。

`_get_current_model_name_from_db`: 若 `settings` 里存的模型名在表中已不存在（文件被删），自动清空脏配置，避免界面显示无效当前模型。

`_register_model_record`: 用 `torch.load(..., map_location="cpu")` 读 checkpoint；同名文件已存在则 `update_model`，否则 `create_model`。上传与训练完成后都调用此函数，避免重复逻辑。

操作

继续追加：

```
def _list_models_from_db() -> list[dict[str, Any]]:
    db = _get_db()
    current_name = (db.get_setting(_CURRENT_MODEL_KEY, "") or "").strip()
    items: list[dict[str, Any]] = []
    for row in db.list_models():
        item = _row_to_dict(row)
        item["is_current"] = item.get("name") == current_name
        items.append(item)
    return items

def _get_current_model_name_from_db() -> str | None:
    db = _get_db()
    current_name = (db.get_setting(_CURRENT_MODEL_KEY, "") or "").strip()
    items = _list_models_from_db()
    valid_names = {str(item.get("name") or "").strip() for item in items if
item.get("name")}
    if current_name and current_name in valid_names:
        return current_name
    if current_name:
        db.set_setting(_CURRENT_MODEL_KEY, "")
    if not items:
        return None
    return items[0].get("name")

def _register_model_record(checkpoint_path: str, fallback_arch: str) -> None:
    db = _get_db()
    checkpoint_path = os.path.abspath(checkpoint_path)
    if not os.path.exists(checkpoint_path):
        return

    checkpoint = torch.load(checkpoint_path, map_location="cpu")
    arch = str(checkpoint.get("arch") or fallback_arch or "")
    class_names = checkpoint.get("class_names") or []
    num_classes = checkpoint.get("num_classes")
    if num_classes is None and isinstance(class_names, list):
        num_classes = len(class_names)

    epoch = checkpoint.get("epoch")
    val_acc = checkpoint.get("val_acc")
```

```

size_mb = round(os.path.getsize(checkpoint_path) / (1024 * 1024), 2)
now_text = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
name = os.path.basename(checkpoint_path)
relative_path = _to_relative_project_path(checkpoint_path)

if db.check_model_exists(name):
    db.update_model(name, relative_path, arch, size_mb, num_classes,
epoch, val_acc, now_text)
else:
    db.create_model(name, relative_path, arch, size_mb, num_classes,
epoch, val_acc, now_text, now_text)

```

代码解读

函数	调用时机
<code>_list_models_from_db</code>	刷新列表、拼下拉选项
<code>_get_current_model_name_from_db</code>	识别页默认权重、模型页「当前模型」文本框
<code>_register_model_record</code>	上传成功、训练成功之后

验证

```

python -c "from gradio_app import _list_models_from_db;
print(len(_list_models_from_db()), '个模型')"

```

若已导入 `garbage.sql`，数量应 ≥ 1 。

步骤 1.4 供页面调用的列表接口

本节目标

封装一层「给 Gradio 回调用」的函数：下拉框只要名字列表，模型页要完整表格 + 提示文案 + 下拉 `gr.update`。

实现说明

- `_refresh_weights`：返回 `["a.pth", "b.pth", ...]`，供 `Dropdown(choices=...)` 使用。
- `_models_list`：返回三元组 `(items, 当前模型名, 提示信息)`，供业务层使用。
- `_model_panel_state`：把三元组转成模型页 4 个输出组件各自需要的格式，上传/设置/训练后的 `.then()` 都复用此函数，避免重复代码。

`auth` 参数在本项目模型查询中未做权限细分（页签名称含「管理员」），保留参数是为了与 `callbacks` 字典接口一致，便于以后加权限判断。

操作

继续追加：

```
def _refresh_weights():
    try:
        return [item["name"] for item in _list_models_from_db()]
    except Exception:
        return []

def _get_current_weight():
    try:
        return _get_current_model_name_from_db()
    except Exception:
        weights = _refresh_weights()
        return weights[0] if weights else None

def _models_list(auth):
    try:
        items = _list_models_from_db()
        current = _get_current_model_name_from_db()
        return items, current or "", f"共 {len(items)} 个模型"
    except Exception as exc:
        return [], None, f"加载模型列表失败: {exc}"

def _model_panel_state(items, cur, msg, choices):
    return _models_to_rows(items), cur or "", msg, gr.update(choices=choices,
value=cur or None)
```

验证

```
python -c "from gradio_app import _models_list;
items,cur,msg=_models_list(None); print(msg, '| current=', cur)"
```

第 2 章 上传权重与设置当前模型

步骤 2.1 上传 .pth 并写入数据库

本节目标

用户在页面选择本地 .pth 后，复制到 models/uploaded/，解析 checkpoint 并写入 models 表。

实现说明

1. Gradio `File` 组件返回临时路径，需用 `getattr(file_obj, "name")` 或 `"path"` 取得真实路径。
2. 仅允许 `.pth` 后缀，防止误传其它文件。
3. 复制使用 `shutil.copy2`，若源与目标相同则跳过复制。
4. 即使 `torch.load` 失败（非标准格式），仍尝试用 `arch="uploaded"` 登记，但可能缺少 `val_acc` 等字段。

操作

继续追加：

```
def _models_upload(auth, file_obj):
    if file_obj is None:
        return "请先选择 .pth 文件"

    src_path = getattr(file_obj, "name", None) or getattr(file_obj, "path",
None) or ""
    src_path = str(src_path or "").strip()
    if not src_path or not os.path.exists(src_path):
        return "上传文件不存在或已失效，请重新选择"

    filename = os.path.basename(src_path)
    if not filename.lower().endswith(".pth"):
        return "只支持上传 .pth 模型文件"

    upload_dir = os.path.join(_project_root(), "models", "uploaded")
    os.makedirs(upload_dir, exist_ok=True)
    dest_path = os.path.join(upload_dir, filename)

    if os.path.abspath(src_path) != os.path.abspath(dest_path):
        shutil.copy2(src_path, dest_path)

    arch = "uploaded"
    try:
        checkpoint = torch.load(dest_path, map_location="cpu")
        arch = str(checkpoint.get("arch") or arch)
    except Exception:
        pass

    try:
        _register_model_record(dest_path, arch)
        return f"上传成功: {filename}"
    except Exception as exc:
        return f"上传成功，但写入数据库失败: {exc}"
```

验证

完成第 4 章 UI 后，在页面测试 **MG2**；或暂时用 Python 模拟（需构造文件对象，建议在 UI 完成后测）。

步骤 2.2 设置当前模型

本节目标

把用户在下拉框选中的模型名写入 `settings.current_model`，供识别页与列表 `is_current` 使用。

实现说明

只改 `settings` 表，不移动磁盘文件。必须先 `check_model_exists`，防止写入不存在的模型名导致识别页选到无效项。

操作

继续追加：

```
def _models_set_current(auth, name):
    model_name = (name or "").strip()
    if not model_name:
        return "请选择要设为当前的模型"

    try:
        db = _get_db()
        if not db.check_model_exists(model_name):
            return f"模型不存在: {model_name}"
        db.set_setting(_CURRENT_MODEL_KEY, model_name)
        return f"当前模型已设置为: {model_name}"
    except Exception as exc:
        return f"设置当前模型失败: {exc}"
```

验证

```
python -c "from gradio_app import _get_db;
print(_get_db().get_setting('current_model'))"
```

或在完成 UI 后执行 **MG3**。

第 3 章 模型训练调度

步骤 3.1 训练脚本约定

本节目标

理解 `train/train_resnet50.py` 的输出格式，确保页面能解析进度并在结束时找到权重路径。

实现说明

训练脚本在**每个 epoch** 打印一行：

```
Epoch 001/010 | train_loss=0.xxx | val_loss=0.xxx | val_acc=0.xxx |
best=0.xxx
```

在**验证集准确率创新高**时保存 checkpoint，结束时打印：

```
Saved best checkpoint to: <绝对路径>
>\models\resnet50\resnet50_20260528_123456.pth
```

`_train_model` 依赖这两类输出：前者更新进度条，后者确定入库文件路径。

checkpoint 字典建议包含字段：

字段	用途
<code>arch</code>	写入 <code>models.arch</code>
<code>num_classes</code> / <code>class_names</code>	类别数
<code>epoch</code> / <code>val_acc</code>	表格展示
<code>state_dict</code>	实际权重（入库不解析，仅留文件）

操作

命令行自测（建议 `--epochs 1`，无 GPU 加 `--cpu`）：

```
python train/train_resnet50.py --epochs 1 --batch-size 8 --out-dir
models/resnet50 --run-name doc_test --cpu
```

验证

终端最后一行包含 `Saved best checkpoint to:`，且路径下文件存在。

步骤 3.2 日志解析与清洗函数

本节目标

从子进程 `stdout` 中提取 epoch 进度，并清理 Windows 下 `tqdm` 乱码字符，保证日志文本框可读。

实现说明

- `_parse_training_progress`: 兼容 `Epoch 1/10` 与 `Epoch 1 / 10` 两种格式。
- `_sanitize_training_line`: 去掉 `\ufffd`、连续 `?`，对含进度条特征的行长按 ASCII+中文过滤。

操作

继续追加：

```
def _parse_training_progress(line: str, fallback_total: int) -> tuple[int, int]:
    match = _EPOCH_PROGRESS_RE.search(line or "")
    if not match:
        return 0, fallback_total
    current = match.group(1) or match.group(3) or "0"
    total = match.group(2) or match.group(4) or str(fallback_total)
    try:
        return max(int(current), 0), max(int(total), 1)
    except ValueError:
        return 0, fallback_total

def _sanitize_training_line(line: str) -> str:
    text = (line or "").replace("\r", "").rstrip()
    if not text:
        return ""
    text = text.replace("\ufffd", "")
    text = re.sub(r"\{3,}", "", text)
    if "%" in text and "|" in text and "/" in text:
        text = re.sub(r"^[^x20-^x7E^u4e00-^u9fff]+", "", text)
        text = text.replace("||", "|")
    return text.strip()
```

验证

```
python -c "from gradio_app import _parse_training_progress;
print(_parse_training_progress('Epoch 2/10 | x', 10))"
```

预期： `(2, 10)`。

步骤 3.3 实现 `_train_model` (subprocess + yield)

本节目标

在 Gradio 回调中启动训练子进程，实时 `yield` 日志、状态、进度；成功后自动入库并设为当前模型。

实现说明

流程分五段：

1. 参数校验：`model_name` 映射 `train_resnet50.py` 等；组装 `python -u script.py --out-dir ...`。
2. 启动进程：`Popen(..., stdout=PIPE, stderr=STDOUT)`，环境变量设 UTF-8。
3. 循环读输出：每行 `yield` 一次，Gradio 刷新三个输出组件。
4. 进程结束：`wait()` 读退出码；为 0 则 `_register_model_record` + `set_setting`。
5. 最后一次 `yield`：显示「训练完成」或失败原因。

为何用生成器（`yield`）？

Gradio 支持生成器回调，每 `yield` 一次就更新界面，实现「边训练边看日志」。若只用 `return`，用户要等训练结束才能看到输出。

为何用 `subprocess`？

训练可能数十分钟，且会占满 GPU；放子进程里即使崩溃也不拖垮 Web 服务，还可随时扩展为「取消训练」。

操作

将下列完整函数追加到 `gradio_app.py`（与仓库 `gradio_app.py` 中 `_train_model` 保持一致）：

```
def _train_model(auth, model_name, save_path, epochs, batch_size, lr,
weight_decay):
    model_name = (model_name or "").strip().lower()
    save_path = (save_path or "").strip() or _model_output_dir(model_name)
    total_epochs = max(int(epochs or 1), 1)
    script_map = {
        "resnet50": "train_resnet50.py",
        "alexnet": "train_alexnet.py",
        "cnn": "train_cnn.py",
    }
    script_name = script_map.get(model_name)
    if not script_name:
        yield "请选择要训练的模型。", "未开始", 0
        return

    train_dir = os.path.join(_project_root(), "train")
    script_path = os.path.join(train_dir, script_name)
    if not os.path.exists(script_path):
        yield f"未找到训练脚本: {script_path}", "训练脚本不存在", 0
        return

    os.makedirs(save_path, exist_ok=True)
    run_name = f"{model_name}_{datetime.now().strftime('%Y%m%d_%H%M%S')}"

    cmd = [
        sys.executable, "-u", script_path,
        "--out-dir", save_path,
        "--epochs", str(total_epochs),
```

```

        "--batch-size", str(int(batch_size or 8)),
        "--lr", str(float(lr or 1e-3)),
        "--weight-decay", str(float(weight_decay or 1e-4)),
        "--num-workers", str(_TRAIN_NUM_WORKERS),
        "--run-name", run_name,
    ]
    if _TRAIN_FORCE_CPU:
        cmd.append("--cpu")
    if model_name == "resnet50" and _TRAIN_RESNET50_PRETRAINED:
        cmd.append("--pretrained")

    yield "开始训练...\n" + " ".join(cmd), "正在启动训练进程", 0

    child_env = os.environ.copy()
    child_env["PYTHONIOENCODING"] = "utf-8"
    child_env["PYTHONUTF8"] = "1"
    child_env["TQDM_ASCII"] = "1"

    process = subprocess.Popen(
        cmd,
        cwd=_project_root(),
        env=child_env,
        stdout=subprocess.PIPE,
        stderr=subprocess.STDOUT,
        text=True,
        encoding=locale.getpreferredencoding(False) or "utf-8",
        errors="replace",
        bufsize=1,
    )

    logs: list[str] = []
    progress_value = 0
    status_text = "训练中..."
    saved_checkpoint_path = ""
    assert process.stdout is not None
    for line in process.stdout:
        clean_line = _sanitize_training_line(line)
        if not clean_line:
            continue
        logs.append(clean_line)
        if clean_line.startswith("Saved best checkpoint to:"):
            saved_checkpoint_path = clean_line.split(":", 1)[1].strip()
        current_epoch, parsed_total = _parse_training_progress(clean_line,
total_epochs)
        if current_epoch > 0:
            progress_value = min(int(current_epoch * 100 / max(parsed_total,
1)), 100)
            status_text = f"训练中: 第 {current_epoch}/{parsed_total} 轮"
        else:
            status_text = clean_line
    yield "\n".join(logs[-400:]), status_text, progress_value

```

```

return_code = process.wait()
if return_code == 0:
    if not saved_checkpoint_path:
        saved_checkpoint_path = os.path.join(save_path, f"
{run_name}.pth")
    try:
        _register_model_record(saved_checkpoint_path, model_name)
        _get_db().set_setting(_CURRENT_MODEL_KEY,
os.path.basename(saved_checkpoint_path))
    except Exception as exc:
        logs.append(f"[数据库写入失败] {exc}")
    logs.append("[训练完成]")
    status_text = "训练完成"
    progress_value = 100
else:
    logs.append(f"[训练失败] 退出码: {return_code}")
    status_text = f"训练失败 (退出码 {return_code}) "
yield "\n".join(logs[-400:]), status_text, progress_value

```

代码解读（关键行）

代码片段	作用
<code>sys.executable, "-u", script_path</code>	用当前 Python 解释器、无缓冲模式跑脚本
<code>cwd=_project_root()</code>	保证脚本里相对路径 <code>data/</code> 能找对
<code>logs[-400:]</code>	只保留最后 400 行，防止日志过长卡 UI
<code>return_code == 0</code>	非 0 表示训练脚本报错，不写入模型表

验证

完成第 4 章后执行 **MG5**、**MG6**；机房无 GPU 可先将 `_TRAIN_FORCE_CPU = True`。

第 4 章 界面与事件绑定

本章把第 1-3 章的后端函数挂到 Gradio 组件上。请将登录章的精简 `build_main_panel` 扩展为仓库中的完整多页签版本（含图像识别、识别记录等），至少补上模型管理页签。

步骤 4.1 扩展 `build_main_panel` 中的「模型管理」页签

本节目标

在 `with gr.Tab("模型管理 (管理员)")`：内放置列表、上传、设当前模型、训练面板，并绑定 `.click().then()` 刷新链。

实现说明

组件布局（自上而下）：

1. 刷新按钮 + 模型表格 + 当前模型文本框 + 提示框
2. 上传文件 + 上传按钮
3. 设置当前模型下拉框 + 设置按钮
4. 训练参数（模型类型、保存目录、epoch、batch、lr 等） + 开始训练 + 日志/状态/进度条

`_models_refresh` 写在页签内部：可访问 `callbacks` 与 `auth_state`，一次刷新 4 个输出。

`.then(_models_refresh, ...)` 模式：上传、设置、训练结束后自动刷新列表，无需用户再点「刷新」。

操作

在 `build_main_panel` 的 `with gr.Tabs()`：内增加（其它页签见仓库完整代码）：

```
with gr.Tab("模型管理（管理员）"):  
    models_refresh = gr.Button("刷新模型列表")  
    models_table = gr.Dataframe(  
        label="模型列表",  
        headers=["name", "size_mb", "num_classes", "epoch", "val_acc",  
"updated_at", "is_current"],  
        interactive=False,  
    )  
    current_model = gr.Textbox(label="当前模型", interactive=False)  
    models_msg = gr.Textbox(label="提示", interactive=False)  
    upload_file = gr.File(label="上传 .pth 权重")  
    upload_btn = gr.Button("上传")  
    upload_msg = gr.Textbox(label="上传提示", interactive=False)  
    set_current_dd = gr.Dropdown(label="设置当前模型",  
choices=callbacks["refresh_weights"]())  
    set_current_btn = gr.Button("设置")  
    set_current_msg = gr.Textbox(label="设置提示", interactive=False)  
  
    gr.Markdown("### 模型训练")  
    with gr.Accordion("训练模型面板", open=True):  
        train_model_name = gr.Dropdown(label="选择模型", choices=["resnet50",  
"alexnet", "cnn"], value="resnet50")  
        train_save_path = gr.Textbox(label="模型保存目录",  
value=_model_output_dir("resnet50"))  
        with gr.Row():  
            train_epochs = gr.Number(label="训练轮数", value=10, precision=0)  
            train_batch_size = gr.Number(label="批大小", value=32,  
precision=0)  
        with gr.Row():  
            train_lr = gr.Number(label="学习率", value=0.001, precision=6)  
            train_weight_decay = gr.Number(label="权重衰减", value=0.0001,  
precision=6)  
        train_btn = gr.Button("开始训练", variant="primary")
```

```

        train_status = gr.Textbox(label="训练状态", value="未开始",
interactive=False)
        train_progress = gr.Slider(label="训练进度", minimum=0, maximum=100,
step=1, value=0, interactive=False)
        train_log = gr.Textbox(label="训练日志", lines=18, max_lines=24,
interactive=False)

def _models_refresh(auth):
    items, cur, msg = callbacks["models_list"](auth)
    choices = callbacks["refresh_weights]()
    return _model_panel_state(items, cur, msg, choices)

def _update_train_save_path(model_name):
    return _model_output_dir((model_name or "resnet50").strip().lower())

models_refresh.click(_models_refresh, inputs=[auth_state], outputs=
[models_table, current_model, models_msg, set_current_dd])
upload_btn.click(callbacks["models_upload"], inputs=[auth_state,
upload_file], outputs=[upload_msg]).then(
    _models_refresh, inputs=[auth_state], outputs=[models_table,
current_model, models_msg, set_current_dd],
)
set_current_btn.click(callbacks["models_set_current"], inputs=
[auth_state, set_current_dd], outputs=[set_current_msg]).then(
    _models_refresh, inputs=[auth_state], outputs=[models_table,
current_model, models_msg, set_current_dd],
)
train_model_name.change(_update_train_save_path, inputs=
[train_model_name], outputs=[train_save_path])
train_btn.click(
    callbacks["train_model"],
    inputs=[auth_state, train_model_name, train_save_path, train_epochs,
train_batch_size, train_lr, train_weight_decay],
    outputs=[train_log, train_status, train_progress],
).then(
    _models_refresh, inputs=[auth_state], outputs=[models_table,
current_model, models_msg, set_current_dd],
)

```

`build_main_panel` 返回值中必须包含（供 `build_app` 绑定）：

```
"models_outputs": [models_table, current_model, models_msg, set_current_dd],
```

验证

启动 `python gradio_app.py`，`admin` 登录后能看到「模型管理（管理员）」页签及上述控件（MG1 目视检查）。

步骤 4.2 更新 `build_app`：注册 `callbacks` 与登录后加载

本节目标

把模型相关函数放进 `callbacks` 字典；登录成功后自动填充模型列表；图像识别页权重下拉与模型列表同源。

实现说明

1. `build_main_panel(auth_state, callbacks)`
第二个参数为函数字典，避免 `build_main_panel` 与业务逻辑耦合，也方便预览模式替换实现。
2. `_load_models_ui`
挂在登录按钮 `.then()` 链上，用户一进主界面就能看到模型表，不必先点开模型页再刷新。
3. 图像识别页

```
rec_weight = gr.Dropdown(choices=callbacks["refresh_weights"](),
value=callbacks["get_current_weight"]())
```

「刷新权重列表」调用 `_weights_update()`，内部同样读 `settings.current_model`。
4. `demo.load` 预渲染主面板（登录章已加）
与模型管理无直接关系，但能避免首次登录主界面空白，请保留。

操作

在 `build_app` 中：

```
def _load_models_ui(auth):
    if not auth:
        return [], "", "请先登录", gr.update()
    try:
        items, cur, msg = _models_list(auth)
        return _model_panel_state(items, cur, msg, _refresh_weights())
    except Exception as exc:
        return [], "", f"加载模型列表失败: {exc}", gr.update()

main_ui = build_main_panel(
    auth_state,
    {
        "refresh_weights": _refresh_weights,
        "get_current_weight": _get_current_weight,
        "models_list": _models_list,
        "models_set_current": _models_set_current,
        "models_upload": _models_upload,
        "train_model": _train_model,
        # ... 识别、记录、用户等其它回调，见仓库 gradio_app.py
    },
)
```

```
login_ui["login_btn"].click(...).then(...).then(
    _load_models_ui,
    inputs=[auth_state],
    outputs=main_ui["models_outputs"],
)
```

验证

`admin` 登录后，不点击刷新即能在模型管理页看到表格有数据（来自 `garbage.sql`）。

第 5 章 功能测试

5.1 运行

```
python gradio_app.py
```

浏览器访问 `http://localhost:6006`（或环境变量 `PORT` 指定端口）。使用 `admin / 123456` 登录。

5.2 测试用例

编号	操作	预期结果
MG1	点击「刷新模型列表」	表格有数据； <code>is_current</code> 列仅一个为 True
MG2	上传合法 <code>.pth</code>	提示成功； <code>models/uploaded/</code> 有文件；表格多一行
MG3	下拉选模型 → 「设置」	提示成功；「当前模型」更新； <code>is_current</code> 列切换
MG4	「图像识别」→「刷新权重列表」	下拉与模型列表一致；默认项为当前模型
MG5	训练 resnet50, epochs=2, batch=16	日志滚动；进度条变化；状态「训练完成」
MG6	训练结束	列表出现新 <code>.pth</code> ；多为当前模型
MG7	查看 <code>models/resnet50/</code>	存在新 <code>.pth</code> 文件
MG8	执行下方 SQL	<code>models</code> 有新行； <code>current_model</code> 已更新

MySQL 验证：

```
SELECT name, arch, val_acc, epoch, updated_at FROM models ORDER BY id DESC  
LIMIT 5;  
SELECT k, v FROM settings WHERE k = 'current_model';
```